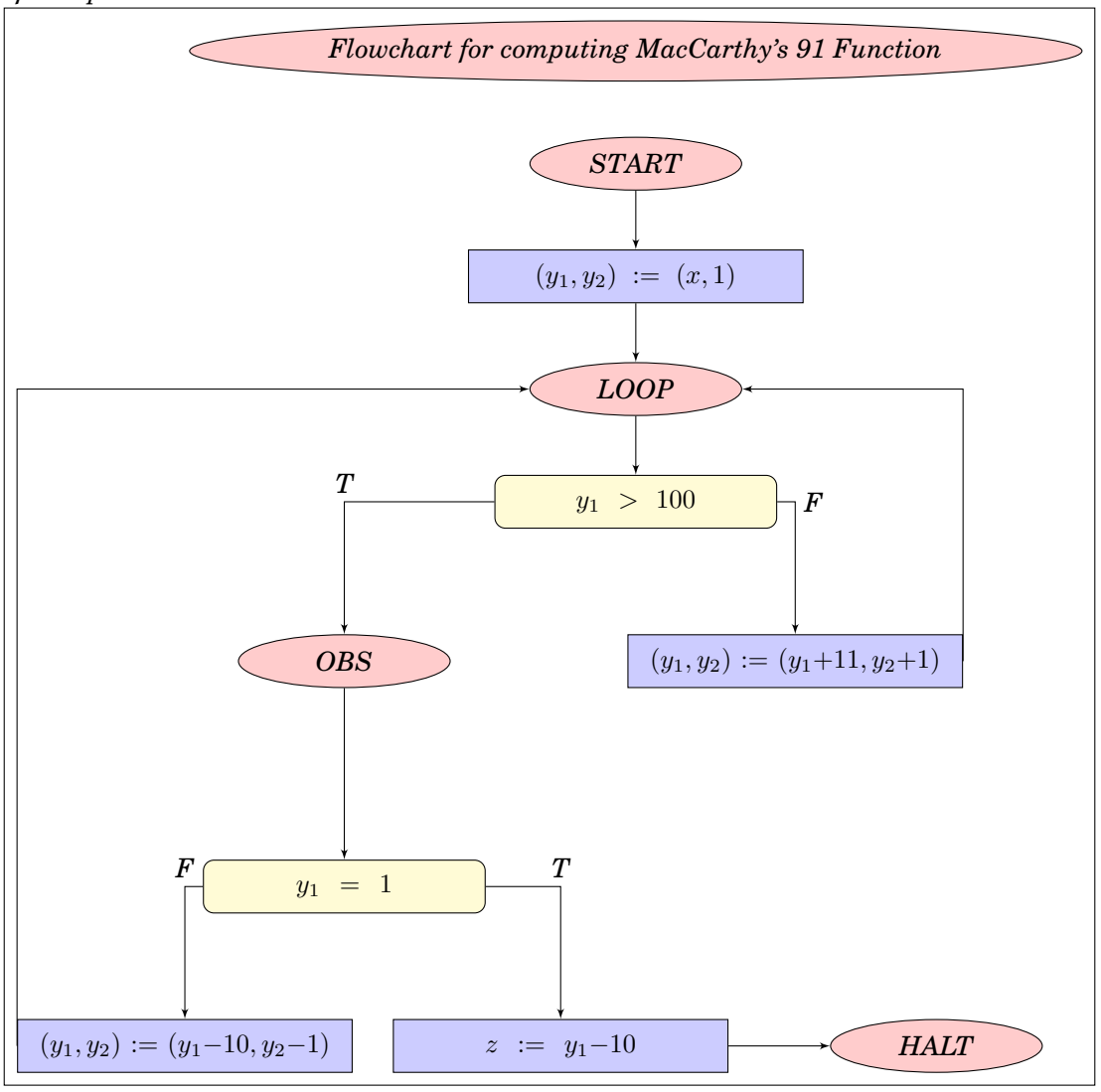
Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation TLA$^+$ (2)
par Dominique Méry
16 octobre 2026

Exercice 1 *appex2_2*
Soit l'organigramme suivant proposé par Z. Manna dans son ouvrage Mathematical Theory of Computation.

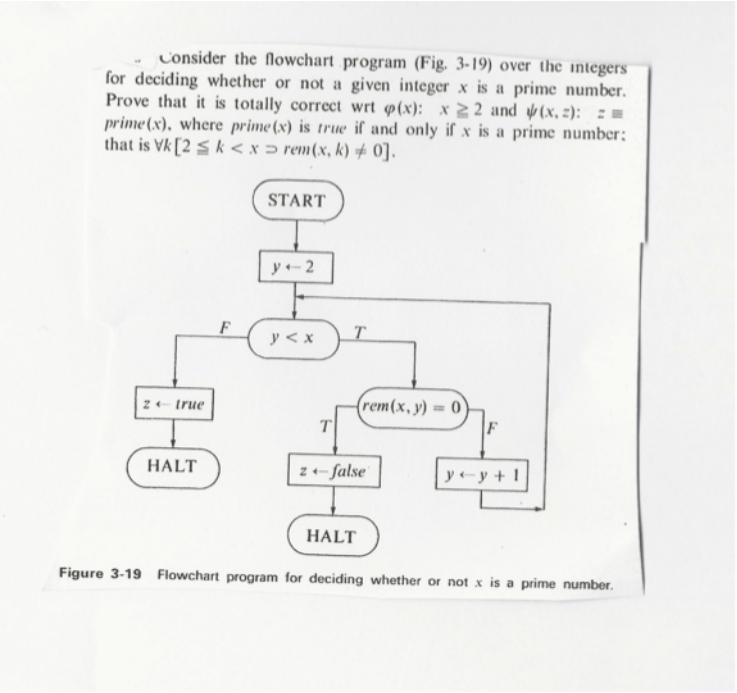


Question 1.1 *Construire un module TLA$^+$ modélisant les différents pas de calcul.*

Question 1.2 *Evaluer l'algorithme en posant des questions de sûreté suivantes :*

1. *l'algorithme est partiellement correct.*
2. *l'algorithme n'a pas d'erreurs à l'exécution.*

Exercice 2 *appex2_3*
Soit le schéma suivant définissant un calcul déterminant sir un nombre entier naturel est premier ou non.



Question 2.1 *Ecrire un modèle TLA modélisant ce schéma de calcul.*

Question 2.2 *Ecrire un invariant à partir d'annotations que vous définirez après avoir défini des points de contrôle.*

Question 2.3 *Vérifier la correction partielle*

Question 2.4 *Vérifier l'absence d'erreurs à l'exécution.*

Exercice 3 *appex2_4*
Le module truc permet de résoudre un problème très classique en informatique : trouver un chemin entre un sommet input et des sommets output supposés être des sommets de sortie.

Question 3.1 *Pour trouver un chemin de input à l'un des sommets de output, il faut poser une question de sûreté à notre système de vérification. Donner une question de sûreté à poser permettant de trouver un chemin de input vers un sommet de output.*

Question 3.2 *On désire utiliser cette technique pour trouver un chemin dans un labyrinthe. Un labyrinthe est représenté par une matrice carrée de taille n. On définit ensuite pour chaque élément* $<< i, j >>$ *de la matrice les voisins communiquant à l'aide de la fonction lab qui associe à* $<< i, j >>$ *les éléments qui peuvent être atteints en un coup. Par exemple, le mouvement possible à partir de* $<< 1, 1 >>$ *est* $<< 2, 1 >>$*, ou le mouvement possible à partir de* $<< 2, 1 >>$ *est* $<< 2, 2 >>$ *ou* $<< 1, 1 >>$*, ou le mouvement possible à partir de* $<< 2, 2 >>$ *est* $<< 2, 3 >>$ *ou* $<< 3, 2 >>$ *ou* $<< 2, 1 >>$*, ...*

```
lab == [<<x,y>> \in (nodes \X nodes) |->
                    IF x=1 /\ y=1  THEN {<<2,1>>} ELSE
                    IF x=2 /\ y=1  THEN {<<2,2>>}
                    IF x=1 /\ y=2  THEN {} ELSE
                    IF x=2 /\ y=2  THEN {<<3,2>>,<<2,3>>} ELSE
                    ELSE {}

                    ]
```

MODULE *Naturals*

A dummy module that defines the operators that are defined by the real Naturals module.

$Nat \triangleq \{\ \}$
$a+b \triangleq \{a,\ b\}$
$a-b \triangleq \{a,\ b\}$
$a \cdot b \triangleq \{a,\ b\}$
$a^b \triangleq \{a,\ b\}$
$a < b \triangleq a = b$
$a > b \triangleq a = b$
$a \leq b \triangleq a = b$
$a \geq b \triangleq a = b$
$a \,\%\, b \triangleq \{a,\ b\}$
$a \div b \triangleq \{a,\ b\}$
$a \,..\, b \triangleq \{a,\ b\}$

MODULE *TLC*

LOCAL INSTANCE *Naturals*
LOCAL INSTANCE *Sequences*

$Print(out,\ val) \triangleq val$
$PrintT(out) \triangleq TRUE$
$Assert(val,\ out) \triangleq$ IF $val = TRUE$ THEN $TRUE$
ELSE CHOOSE $v : TRUE$
$JavaTime \triangleq$ CHOOSE $n : n \in Nat$
$TLCGet(i) \triangleq$ CHOOSE $n : TRUE$
$TLCSet(i,\ v) \triangleq TRUE$

$d :> e \triangleq [x \in \{d\} \mapsto e]$
$f \ @@\ g \triangleq [x \in (\text{DOMAIN } f) \cup (\text{DOMAIN } g) \mapsto$
IF $x \in$ DOMAIN f THEN $f[x]$ ELSE $g[x]]$
$Permutations(S) \triangleq$
$\{f \in [S \to S] : \forall w \in S : \exists v \in S : f[v] = w\}$

In the following definition, we use Op as the formal parameter rather than \prec because TLC Version 1 can't handle infix formal parameters.

$SortSeq(s,\ Op(_,\ _)) \triangleq$
LET $Perm \triangleq$ CHOOSE $p \in Permutations(1 \,..\, Len(s)) :$
$\forall i,\ j \in 1..Len(s) :$
$(i < j) \Rightarrow Op(s[p[i]],\ s[p[j]]) \vee (s[p[i]] = s[p[j]])$
IN $[i \in 1..Len(s) \mapsto s[Perm[i]]]$

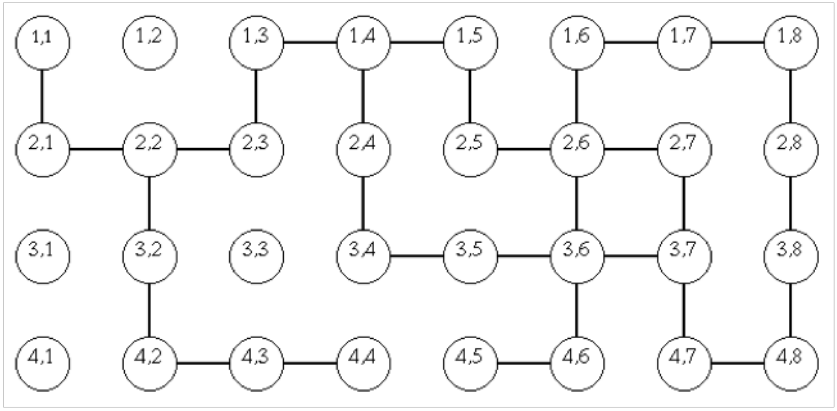
$RandomElement(s) \triangleq$ CHOOSE $x \in s : TRUE$

$Any \triangleq$ CHOOSE $x : TRUE$

$ToString(v) \triangleq (\text{CHOOSE } x \in [a : v,\ b : STRING] : TRUE).b$

$TLCEval(v) \triangleq v$

FIGURE 1 – Modules Naturals.tla et TLC.tla



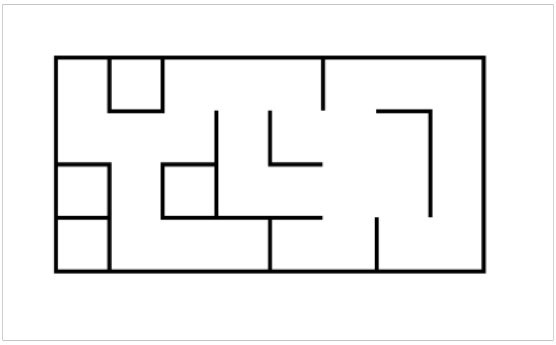


FIGURE 2 – Labyrinthe

Modifier le module truc pour traiter ce problème et donner la question à poser pour trouver une sortie.

MODULE *truc*

EXTENDS *Integers*, *TLC*
VARIABLES p
CONSTANTS *input*, *output*

$n \triangleq 10$
nodes $\triangleq 1..n$
$l \triangleq [i \in 1..n \mapsto$ IF $i = 1$ THEN $\{4, 5\}$ ELSE
IF $i = 2$ THEN $\{6, 7, 10\}$ ELSE
IF $i = 4$ THEN $\{7, 8\}$ ELSE
IF $i = 5$ THEN $\{\}$ ELSE
IF $i = 6$ THEN $\{4\}$ ELSE
IF $i = 7$ THEN $\{5\}$ ELSE
IF $i = 8$ THEN $\{5, 2\}$ ELSE
$\{\}$
$]$

Init $\triangleq p = 1$
$M(i) \triangleq \wedge i \in l[p]$
$\wedge p' = i$

$$\mathit{Next} \triangleq \exists\, i \in 1..n : M(i)$$